

Lembar Materi & Praktikum: File Logging di Gin dan Dasar Unit Testing di Golang

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Praktikum ini melengkapi materi teori pada [18. validated-api.md](#). Anda akan dipandu untuk mengimplementasikan sistem penanganan log terstruktur tingkat lanjut (*production-grade logging*) dan pengujian unit (*unit testing*) pada proyek **Sistem Tiket Konser** (`go-tiket-konser`) menggunakan framework **Gin**. Kita akan belajar memisahkan output log ke dalam **berkas fisik (.log)** menggunakan **Logrus**, membuat **custom logging middleware** untuk HTTP requests, serta memprogram pengujian logika bisnis secara otomatis menggunakan paket bawaan **testing Go** dan pustaka asersi **Testify**.

Sesi 1: Teori & Urgensi Logging Tingkat Produksi (45 Menit)

1. Mengapa `fmt.Println` adalah Dosa Besar di Server Produksi?

Sebagai developer pemula, kita terbiasa menggunakan perintah `fmt.Println()` atau `log.Println()` bawaan Go untuk melacak jalannya program. Namun, di dunia industri (*production level*), hal ini dilarang keras karena:

- **Tidak Memiliki Level (Severity):** Kita tidak bisa membedakan mana catatan biasa (`INFO`), peringatan kinerja sistem (`WARNING`), kesalahan fungsionalitas (`ERROR`), atau kesalahan fatal yang membuat server lumpuh (`FATAL`).
- **Tidak Terstruktur:** Output berupa teks biasa sangat sulit di-parsing oleh mesin pencari log modern seperti Elasticsearch, Splunk, atau Datadog.
- **Hanya Muncul di Terminal:** Jika aplikasi di-restart atau dideploy dalam kontainer Docker, semua log terminal sebelumnya akan hilang selamanya. Kita wajib mencatat log ke **Berkas Fisik (File)** agar terdokumentasi secara permanen.

2. Klasifikasi Log Levels

Sistem logging terstruktur membagi peristiwa sistem menjadi beberapa tingkat kepentingan:

1. **DEBUG:** Informasi sangat mendetail yang hanya berguna selama proses pengembangan lokal.
2. **INFO:** Peristiwa umum yang menunjukkan aplikasi berjalan normal (misal: "Database sukses terkoneksi", "Server berjalan di port 8080").

3. **WARN**: Kejadian janggal yang belum merusak sistem tetapi berpotensi memicu error (misal: "Waktu respons database lambat > 2 detik", "Ada percobaan login gagal").
4. **ERROR**: Terjadi kesalahan operasional (misal: "Gagal mengambil data dari database"), namun aplikasi masih bisa hidup melayani pengguna lain.
5. **FATAL**: Kesalahan kritis yang menyebabkan aplikasi mati seketika. Biasanya memicu sistem operasi untuk keluar (`os.Exit(1)`).

3. Pengenalan Logrus

Logrus (github.com/sirupsen/logrus) adalah library logging terstruktur terpopuler di komunitas Go. Ia mendukung penulisan log dalam format JSON dan memisahkan aliran keluaran secara paralel (misalnya dicetak ke layar sekaligus ditulis ke file log).

Sesi 2: Menulis Log ke File & Middleware Logging di Gin (45 Menit)

1. Menggunakan `io.MultiWriter`

Di server VPS produksi, administrator ingin agar log tetap terlihat saat memantau terminal (*Stdout*), namun di saat bersamaan, data log tersebut harus ditulis ke dalam file berkas (misal: `logs/app.log`). Go menyediakan utilitas `io.MultiWriter` untuk menggabungkan beberapa target penulisan berkas menjadi satu aliran log terpadu.

2. Rotasi Log (Log Rotation) & Retensi 7 Hari

Menulis log ke dalam satu file tunggal terus-menerus memiliki risiko fatal: **kehabisan kapasitas penyimpanan disk (disk out of space)**. Pada aplikasi produksi berskala besar, file log dapat membengkak hingga puluhan Gigabyte hanya dalam hitungan minggu.

Solusinya adalah Log Rotation:

- File log dirotasi (dipecah) setelah mencapai ukuran tertentu (misalnya setiap file mencapai 10MB).
- File log lama dikompresi (misalnya menjadi format `.gz`) untuk menghemat ruang penyimpanan.
- **Retensi Log (Retention)**: Menyimpan file log lama hanya untuk jangka waktu tertentu, misalnya **7 hari terakhir**, dan secara otomatis menghapus log yang lebih tua dari itu.

Di Go, standar industri untuk menangani rotasi log secara otomatis adalah pustaka **Lumberjack** (gopkg.in/natefinch/lumberjack.v2). Lumberjack mengimplementasikan interface `io.Writer` ,

sehingga dapat digunakan secara langsung sebagai pengganti file log mentah pada `io.MultiWriter`.

3. Custom Logging Middleware di Gin

Gin secara default membawa logger berwarna. Namun, format bawaan tersebut tidak terstruktur. Kita akan merancang sebuah middleware kustom yang mencegat setiap request HTTP masuk, lalu mencatat:

- Waktu request diterima (`time`).
- IP Address client (`client_ip`).
- HTTP Method & Request Path (`method` , `path`).
- Kode HTTP Response status (`status`).
- Durasi waktu pemrosesan (*latency*).
- Pesan error (jika ada).

Format log ini akan ditulis dalam bentuk JSON satu baris (*one-line JSON*) ke berkas log.

Sesi 3: Teori Dasar Unit Testing di Golang (45 Menit)

1. Mengapa Unit Testing Wajib bagi Backend Engineer?

- **Mencegah Regresi (Regression):** Menjamin bahwa fitur lama tidak rusak ketika Anda menambahkan fitur baru atau melakukan perbaikan kode (*refactoring*).
- **Dokumentasi Hidup:** Kode test menjelaskan bagaimana sebuah fungsi seharusnya berperilaku dengan berbagai variasi input data.
- **Meningkatkan Kualitas Kode:** Menulis test memaksa developer untuk membuat kode yang terdekopel (*decoupled*) agar mudah diuji secara modular.

2. Aturan Penulisan Unit Test di Go

Go memiliki pustaka pengujian bawaan yang sangat ketat. Aturan mainnya:

1. **Nama File:** Wajib diakhiri dengan sufiks `_test.go`. Misal, pengujian untuk `service/concert.go` harus ditarik di file `service/concert_test.go`.
2. **Nama Fungsi:** Wajib diawali huruf kapital `Test` diikuti nama fungsinya. Harus menerima argumen pointer (`t *testing.T`). Contoh: `func TestValidateConcertDate(t *testing.T)`.
3. **Package:** Berada dalam package yang sama dengan file yang diuji.

3. Library Asersi Testify

Secara bawaan, pengujian Go menggunakan kondisional `if` manual untuk melaporkan kegagalan:

```
if hasil ≠ ekspektasi {  
    t.Errorf("Salah! Hasilnya %d, harusnya %d", hasil, ekspektasi)  
}
```

Untuk mempermudah penulisan asersi, industri Go menggunakan library **Testify** (github.com/stretchr/testify/assert) untuk mempersingkat baris penulisan asersi:

```
assert.Equal(t, ekspektasi, hasil, "Nilai kembalian harus cocok")
```

Sesi 4: Praktikum Mandiri – Langkah Implementasi File Logging & Unit Test (45 Menit)

Mari pelajari cara menerapkan integrasi fungsionalitas ini pada proyek **Sistem Tiket Konser** Anda.

1. Membuat Pusat Logger Logrus dengan Log Rotation:

[pkg/logger/logger.go](#)

Sebelum membuat berkas logger, unduh terlebih dahulu pustaka **Lumberjack** untuk penanganan rotasi log otomatis:

```
go get -u gopkg.in/natefinch/lumberjack.v2
```

Buat folder baru `pkg/logger/` dan buat file `logger.go` untuk inisialisasi Logrus yang terintegrasi dengan rotasi log otomatis dan retensi 7 hari:

```
package logger  
  
import (  
    "io"  
    "os"  
    "path/filepath"  
  
    "github.com/sirupsen/logrus"  
    "gopkg.in/natefinch/lumberjack.v2"  
)  
  
var Log *logrus.Logger  
  
// InitLogger mengonfigurasi logrus agar menulis ke terminal dan logs/app.log dengan  
// rotasi log otomatis  
func InitLogger() {  
    Log = logrus.New()
```

```

// 1. Set format ke JSON agar mudah dibaca oleh parser/mesin monitoring
Log.SetFormatter(&logrus.JSONFormatter{
    TimestampFormat: "2006-01-02 15:04:05",
})

// 2. Buat folder logs jika belum terbentuk
logDir := "logs"
_ = os.MkdirAll(logDir, os.ModePerm)

// 3. Konfigurasi Log Rotation dengan Lumberjack
lumberjackLogger := &lumberjack.Logger{
    Filename:   filepath.Join(logDir, "app.log"),
    MaxSize:    10, // Ukuran maksimum berkas log (dalam Megabyte)
    sebelum dirotasi
    MaxBackups: 3, // Jumlah maksimum arsip berkas log lama yang
    disimpan
    MaxAge:     7, // Masa simpan log lama (dalam hari). Log > 7 hari
    akan dihapus otomatis
    Compress:   true, // Kompresi berkas log cadangan menjadi format .gz
    LocalTime:  true, // Menggunakan waktu lokal server untuk penulisan
    timestamp cadangan
}

// 4. io.MultiWriter mengalirkan data log ke layar terminal (os.Stdout) dan
Lumberjack secara paralel
mw := io.MultiWriter(os.Stdout, lumberjackLogger)
Log.SetOutput(mw)

// Set level default
Log.SetLevel(logrus.InfoLevel)
}

```

2. Merancang Custom Logger Middleware Gin: [middleware/logger.go](#)

Buat berkas baru bernama `logger.go` di bawah folder `middleware/` untuk merekam detail transaksi HTTP request:

```

package middleware

import (
    "time"

    "go-tiket-konser/pkg/logger"

    "github.com/gin-gonic/gin"
    "github.com/sirupsen/logrus"
)

// LoggerMiddleware mencegat respons Gin dan mencatat metriknya ke berkas Logrus

```

```

func LoggerMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        startTime := time.Now()

        // Biarkan request berjalan ke handler berikutnya
        c.Next()

        endTime := time.Now()
        latency := endTime.Sub(startTime)

        // Ambil data log
        statusCode := c.Writer.Status()
        clientIP := c.ClientIP()
        method := c.Request.Method
        path := c.Request.URL.Path

        logFields := logrus.Fields{
            "status":      statusCode,
            "latency_ms":   latency.Milliseconds(),
            "client_ip":    clientIP,
            "method":       method,
            "path":          path,
        }

        // Jika status HTTP ≥ 500 (Kesalahan Server), catat dengan level
Error
        if statusCode ≥ 500 {
            logger.Log.WithFields(logFields).Error("Terjadi kesalahan
server internal pada route ini")
        } else if statusCode ≥ 400 {
            // Jika status HTTP 4xx (Kesalahan Input Klien), catat dengan
level Warn
            logger.Log.WithFields(logFields).Warn("Permintaan klien tidak
valid atau otorisasi gagal")
        } else {
            // Jika sukses 2xx / 3xx, catat dengan level Info
            logger.Log.WithFields(logFields).Info("HTTP Request sukses
diproses")
        }
    }
}

```

3. Modifikasi Main untuk Inisialisasi Logger: [main.go](#)

Inisialisasi logger di awal start aplikasi dan daftarkan middleware kustom Anda di router.

```

package main

import (
    "go-tiket-konser/config"
    "go-tiket-konser/pkg/logger"

```

```

        "go-tiket-konser/routes"
    )

    func main() {
        // 1. Inisialisasi sistem logging
        logger.InitLogger()
        logger.Log.Info("Memulai startup server...")

        // 2. Inisialisasi database & router
        config.InitDB()
        r := routes.SetupRouter(config.DB)

        logger.Log.Info("Server berjalan di port :8080")
        _ = r.Run(":8080")
    }

```

Di dalam `routes/routes.go`, pasang middleware kustom:

```

func SetupRouter(db *gorm.DB) *gin.Engine {
    r := gin.New() // Gunakan gin.New() agar tidak memuat logger bawaan gin
    r.Use(gin.Recovery())
    r.Use(middleware.LoggerMiddleware()) // Pasang Custom Logger
    // ... (setup routes) ...
}

```

3b. Implementasi Logging Terstruktur pada Alur Auth (Login, Register, OTP) & Concert: [handler/auth.go] & [handler/concert.go]

Untuk memantau aktivitas transaksi kritikal, gunakan logger terstruktur yang sudah diinisiasi di dalam handler:

1. Aktivitas Register & Login: [handler/auth.go]

```

// Di dalam AuthHandler.Register saat user baru berhasil terdaftar:
logger.Log.WithFields(logrus.Fields{
    "email":      newUser.Email,
    "client_ip":  c.ClientIP(),
    "action":     "auth_register_success",
}).Info("User baru berhasil terdaftar")

// Di dalam AuthHandler.Login saat terjadi kegagalan login (salah password):
logger.Log.WithFields(logrus.Fields{
    "email":      req.Email,
    "client_ip":  c.ClientIP(),
    "action":     "auth_login_failed",
}).Warn("Percobaan login gagal: kata sandi tidak cocok")

```

2. Aktivitas Verifikasi OTP: [handler/auth.go]

```
// Di dalam AuthHandler.VerifyOTP saat kode OTP sukses diverifikasi:
logger.Log.WithFields(logrus.Fields{
    "email":      req.Email,
    "client_ip":  c.ClientIP(),
    "action":     "auth_otp_verified",
}).Info("Kode OTP sukses diverifikasi, token JWT berhasil diterbitkan")
```

3. Aktivitas Pengelolaan Konser: [handler/concert.go]

```
// Di dalam ConcertHandler.CreateConcert saat konser baru berhasil disimpan ke
database:
logger.Log.WithFields(logrus.Fields{
    "concert_id": newConcert.ID,
    "title":      newConcert.Title,
    "venue":      newConcert.Venue,
    "action":     "concert_create_success",
}).Info("Konser baru berhasil ditambahkan ke katalog")
```

4. Membuat Unit Test Bisnis Validasi Konser:

[service/concert_validation_test.go](#)

Kita akan membuat unit test sederhana untuk menguji fungsi bisnis validasi tanggal konser (Konser tidak boleh dijadwalkan di masa lalu). Pertama, pasang library asersi Testify di terminal proyek Anda:

```
go get -u github.com/stretchr/testify
```

Buat file baru `concert_validation_test.go` di bawah folder `service/` :

```
package service

import (
    "testing"
    "time"

    "go-tiket-konser/models"

    "github.com/stretchr/testify/assert"
)

// Helper sederhana fungsi validasi tanggal konser
func IsValidConcertDate(date time.Time) bool {
    return date.After(time.Now())
}

func TestIsValidConcertDate_Success(t *testing.T) {
    // Skenario 1: Tanggal konser di masa depan (2 hari dari sekarang)
    futureDate := time.Now().Add(48 * time.Hour)
```

```

    result := IsValidConcertDate(futureDate)

    // Verifikasi asersi sukses
    assert.True(t, result, "Tanggal di masa depan harus dianggap valid")
}

func TestIsValidConcertDate_Failed_PastDate(t *testing.T) {
    // Skenario 2: Tanggal konser di masa lalu (5 jam yang lalu)
    pastDate := time.Now().Add(-5 * time.Hour)
    result := IsValidConcertDate(pastDate)

    // Verifikasi asersi gagal
    assert.False(t, result, "Tanggal di masa lalu harus dianggap tidak valid")
}

```

5. Membuat Unit Test untuk Auth Login & Layanan Konser: [service/auth_test.go] & [service/concert_test.go]

Untuk menguji logika bisnis secara modular tanpa menyentuh koneksi database asli, kita membuat Mock Repository menggunakan RAM slice.

1. **Unit Test Layanan Auth (Login): [service/auth_test.go]** Buat file `service/auth_test.go` untuk menguji fungsionalitas AuthService.Login pada skenario sukses dan gagal:

```

package service

import (
    "testing"

    "go-tiket-konser/models"

    "github.com/stretchr/testify/assert"
)

type mockUserRepository struct {
    users []models.User
}

func (m *mockUserRepository) FindByEmail(email string) (models.User, error) {
    for _, u := range m.users {
        if u.Email == email {
            return u, nil
        }
    }
    return models.User{}, models.ErrUserNotFound
}

func (m *mockUserRepository) Create(user *models.User) error {
    m.users = append(m.users, *user)
    return nil
}

```

```

}

// Test Login Sukses
func TestAuthService_Login_Success(t *testing.T) {
    mockRepo := &mockUserRepository{
        users: []models.User{
            {ID: 1, Email: "admin@gmail.com", Password: "hashedpassword",
Role: "admin"},
        },
    }

    // Mocking Auth Service (Password verifikasi dicocokkan lewat dummy/helper)
    authService := NewAuthService(mockRepo)
    user, err := authService.Login("admin@gmail.com", "hashedpassword")

    assert.NoError(t, err)
    assert.Equal(t, "admin@gmail.com", user.Email)
    assert.Equal(t, "admin", user.Role)
}

// Test Login Gagal - User Tidak Ditemukan
func TestAuthService_Login_Failed_UserNotFound(t *testing.T) {
    mockRepo := &mockUserRepository{users: []models.User{}}
    authService := NewAuthService(mockRepo)

    _, err := authService.Login("unknown@gmail.com", "password")

    assert.Error(t, err)
    assert.Equal(t, models.ErrUserNotFound, err)
}

```

2. Unit Test Layanan Konser (CreateConcert): [service/concert_test.go] Buat file

service/concert_test.go untuk memverifikasi validasi tanggal aman pembuatan konser (harus minimal H+7 dari hari ini) dan penolakan jika terjadi duplikasi judul:

```

package service

import (
    "errors"
    "testing"
    "time"

    "go-tiket-konser/models"

    "github.com/stretchr/testify/assert"
)

type mockConcertRepository struct {
    concerts []models.Concert
}

```

```

func (m *mockConcertRepository) Create(concert *models.Concert) error {
    m.concerts = append(m.concerts, *concert)
    return nil
}

func (m *mockConcertRepository) FindByID(id int) (models.Concert, error) {
    for _, c := range m.concerts {
        if c.ID == id {
            return c, nil
        }
    }
    return models.Concert{}, errors.New("not found")
}

func (m *mockConcertRepository) Update(concert *models.Concert) error {
    return nil
}

func (m *mockConcertRepository) Delete(id int) error {
    return nil
}

func (m *mockConcertRepository) FindAllPaginated(search string, limit int, offset int,
sort string) ([]models.Concert, int64, error) {
    return m.concerts, int64(len(m.concerts)), nil
}

func (m *mockConcertRepository) FindByTitle(title string) (models.Concert, error) {
    for _, c := range m.concerts {
        if c.Title == title {
            return c, nil
        }
    }
    return models.Concert{}, errors.New("not found")
}

// Test Create Concert Sukses (H+7)
func TestConcertService_CreateConcert_Success(t *testing.T) {
    mockRepo := &mockConcertRepository{}
    service := NewConcertService(mockRepo)

    futureTime := time.Now().Add(10 * 24 * time.Hour) // H+10 (Aman)
    concert := &models.Concert{
        Title:      "Sheila On 7 Live in Jakarta",
        Description: "Konser Musik Terbesar",
        Date:       futureTime,
        Venue:      "ICE BSD",
        Status:     "upcoming",
    }

    err := service.CreateConcert(concert)
    assert.NoError(t, err)
}

```

```
// Test Create Concert Gagal - Tanggal di Masa Lalu
func TestConcertService_CreateConcert_Failed_PastDate(t *testing.T) {
    mockRepo := &mockConcertRepository{}
    service := NewConcertService(mockRepo)

    pastTime := time.Now().Add(-24 * time.Hour) // H-1 (Masa Lalu)
    concert := &models.Concert{
        Title:      "Sheila On 7 Live in Jakarta",
        Description: "Konser Musik Terbesar",
        Date:       pastTime,
        Venue:      "ICE BSD",
        Status:     "upcoming",
    }

    err := service.CreateConcert(concert)
    assert.Error(t, err)
    assert.Contains(t, err.Error(), "tanggal konser harus di masa depan")
}
```

6. Menjalankan Unit Test dan Analisis Coverage

Jalankan pengujian unit dari terminal Anda:

```
# Menjalankan seluruh unit test dengan log mendetail (verbose)
go test -v ./...
```

Menganalisis Kode Coverage (Cakupan Pengujian): Untuk melihat berapa persen (%) baris kode kita yang telah dilindungi oleh unit test:

1. Jalankan profil coverage ke berkas output:

```
go test -coverprofile=cover.out ./...
```

2. Visualisasikan hasil coverage dalam bentuk halaman HTML interaktif di browser Anda:

```
go tool cover -html=cover.out
```

Halaman browser akan menandai baris kode berwarna hijau (teruji) dan berwarna merah (belum teruji).

Sesi 5: Tantangan Praktikum Mandiri (Tugas Mandiri)

Selesaikan tantangan berikut untuk melatih penanganan error terpusat dan implementasi mock testing.

Tantangan 1: Middleware Error Logging yang Menangkap Panic (Panic Recovery Logging)

Jika aplikasi mengalami kegagalan fatal (*runtime error/panic*) di dalam salah satu Handler, aplikasi Gin akan memulihkan server melalui middleware `gin.Recovery()`. Namun, informasi *stack trace* (baris letak crash terjadi) hanya tertulis di konsol terminal dan tidak tercatat di file log.

- **Tugas:**

1. Buatlah middleware kustom baru bernama `PanicRecoveryLogger()` di dalam `middleware/logger.go`.
2. Gunakan blok `defer` dan fungsi `recover()` bawaan Go untuk menangkap sinyal panic.
3. Jika sinyal panic tertangkap, ekstrak informasi error beserta *stack trace* baris kodenya (gunakan pustaka `"runtime/debug"` bawaan Go melalui perintah `debug.Stack()`).
4. Tuliskan log berlevel **FATAL** atau **ERROR** menggunakan Logrus ke dalam file `logs/app.log`.
5. Kembalikan respons JSON seragam ke client dengan format status HTTP 500:

```
{"success": false, "message": "Kesalahan sistem internal yang kritis terjadi."}
```

Tantangan 2: Menulis Unit Test Ter-mocking untuk Service Layer

Lakukan pengujian unit untuk `ConcertService` untuk membuktikan logika bisnis penolakan pembuatan konser dengan nama judul yang sama (*duplicate title validation*).

- **Tugas:**

1. Buatlah file pengujian baru `service/concert_test.go`.
2. Karena kita tidak ingin test mengotori atau menyentuh database PostgreSQL asli, kita harus membuat implementasi **Mock Repository** yang meniru database menggunakan memory slice.
3. Rancang mock struct sederhana yang mengimplementasikan interface `repository.ConcertRepository`:

```
type mockConcertRepository struct {
    concerts []models.Concert
}

func (m *mockConcertRepository) Create(concert *models.Concert) error {
    m.concerts = append(m.concerts, *concert)
    return nil
}

func (m *mockConcertRepository) FindAll() ([]models.Concert, error) {
    return m.concerts, nil
}
```

```
// ... implementasikan method interface repository lainnya (cukup  
kembalikan nil/dummy) ...
```

4. Di dalam fungsi `TestCreateConcert_DuplicateTitle(t *testing.T)`, instansiasi `mockConcertRepository`, suntikkan data awal konser berjudul `"Coldplay Jakarta"`, lalu buat instance `concertService` yang dihubungkan ke mock tersebut.
5. Picu pemanggilan method `service.CreateConcert()` dengan judul yang sama `"Coldplay Jakarta"`, lalu asersikan bahwa error yang keluar adalah `models.ErrConcertAlreadyExists`.